MATERIAL DE ESTUDIO · JS / TS

JavaScript y TypeScript
Guía práctica completa

Explicaciones detalladas, ejemplos de código comentados y ejercicios para dominar JavaScript moderno, el sistema de tipos de TypeScript, estructuras de datos y análisis de complejidad.

Incluye: módulos, closures, async/await, operadores modernos, tipos, genéricos, narrowing, utility types, DOM tipado, tsconfig, validación en runtime, inmutabilidad, estructuras de datos y Big-O.
Incluye soluciones orientativas al final.

Índice

01 JavaScript moderno

02 Operadores modernos

03 Tipos primitivos y especiales

04 Arrays y tuplas

05 Interfaces y type aliases

06 Enums y tipos literales

07 Inferencia y aserciones

08 Funciones y genéricos

09 Uniones e intersecciones

10 Narrowing y type guards

11 Utility types

12 DOM APIs tipadas

13 Configuración del proyecto TS

14 Validación en runtime

15 Errores, inmutabilidad y composición

16 Estructuras de datos

17 Análisis de complejidad

18 Soluciones orientativas

> **Cómo estudiar:** lee cada sección, escribe los ejemplos a mano (o en el editor), luego intenta los ejercicios *sin mirar* la solución. TypeScript se entiende mejor cuando el compilador te corrige: usa `tsc --strict` o el playground de TypeScript.

1. JavaScript moderno

JavaScript moderno (ES2015+) organiza el código en **módulos**, controla el **alcance (scope)** con `let` / `const`, usa **closures** para encapsular estado, y maneja asincronía con **Promises** y `async` / `await`. Además, ofrece APIs potentes para transformar arrays y objetos.

1.1 Módulos (ESM)

Un módulo es un archivo con su propio scope. Exportas lo que quieres compartir e importas lo que necesitas. Esto evita variables globales y permite tree-shaking.

```
// math.js
export const PI = 3.14159;
export function sum(a, b) { return a + b; }
export default function multiply(a, b) { return a * b; }

// app.js
import multiply, { PI, sum } from './math.js';
import * as math from './math.js'; // namespace

console.log(sum(2, 3), multiply(2, PI));
console.log(math.sum(1, 1));
```

Qué pasa: `export default` permite un export "principal" (puedes renombrarlo al importar). Los named exports deben coincidir en nombre (salvo `as`). En Node/navegador moderno usa `"type": "module"` o extensión `.mjs` / atributo `type="module"` en el script.

1.2 Scope: var vs let/const

El *scope* es el alcance donde una variable es visible. `var` es function-scoped y se "eleva" (hoisting) de forma confusa. `let` y `const` son block-scoped (`{ }`).

```
function demoScope() {
  if (true) {
    var a = 1;   // visible en toda la función
    let b = 2;   // solo dentro del if
    const c = 3; // igual que let, pero no se reasigna
  }
  console.log(a); // 1
  // console.log(b); // ReferenceError
}

// Temporal Dead Zone: no uses let/const antes de declararlas
// console.log(x); // Error
let x = 10;

const user = { name: 'Ana' };
user.name = 'Luis'; // OK: mutas la propiedad
// user = {};      // Error: no reasignas la referencia
```

Regla práctica: usa `const` por defecto; `let` solo si necesitas reasignar; evita `var`.

1.3 Closures

Un closure es una función que "recuerda" variables de su entorno léxico aunque ese entorno ya haya terminado de ejecutarse. Sirve para encapsular estado privado y crear factories.

```
function createCounter(start = 0) {
  let count = start; // privada: no accesible desde fuera

  return {
    next() { return ++count; },
    value() { return count; },
  };
}

const c = createCounter(10);
console.log(c.next()); // 11
console.log(c.next()); // 12
console.log(c.value()); // 12
// count no existe fuera: encapsulación
```

Por qué importa: callbacks, hooks, memoización y módulos internos se basan en closures. Cuidado con closures en bucles con `var` (usa `let`).

1.4 Promises

Una Promise representa un valor futuro: `pending` → `fulfilled` o `rejected`. Encadena con `.then` / `.catch` / `.finally`.

```
function fetchUser(id) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (id > 0) resolve({ id, name: 'Ada' });
      else reject(new Error('id inválido'));
    }, 200);
  });
}

fetchUser(1)
  .then(user => {
    console.log(user.name);
    return user.id;
  })
  .then(id => console.log('id:', id))
  .catch(err => console.error(err.message))
  .finally(() => console.log('listo'));

// Combinadores útiles
Promise.all([fetchUser(1), fetchUser(2)]);      // falla si uno falla
Promise.allSettled([fetchUser(1), fetchUser(-1)]); // espera todos
Promise.race([fetchUser(1), fetchUser(2)]);     // el primero en terminar
Promise.any([fetchUser(-1), fetchUser(2)]);     // primer éxito
```

1.5 async / await

Azúcar sintáctico sobre Promises: el código se lee de forma secuencial. `async` siempre devuelve una Promise. Usa `try/catch` para errores.

```
async function loadProfile(id) {
  try {
    const user = await fetchUser(id);
    const posts = await fetchPosts(user.id); // secuencial
    return { user, posts };
  } catch (err) {
    console.error('falló', err);
    throw err; // re-lanzar si el caller debe enterarse
  }
}

// Paralelismo: arranca ambas a la vez
async function loadFaster(id) {
  const userPromise = fetchUser(id);
  const postsPromise = fetchPosts(id);
  const [user, posts] = await Promise.all([userPromise, postsPromise]);
  return { user, posts };
}
```

> **Error común:** olvidar `await` dentro de un bucle cuando necesitas orden, o usar `await` en serie cuando podrías paralelizar con `Promise.all`.

1.6 Operaciones con arrays y objetos

```
const nums = [1, 2, 3, 4, 5];

// Transformar / filtrar / reducir (no mutan el original)
const doubles = nums.map(n => n * 2);
const evens = nums.filter(n => n % 2 === 0);
const total = nums.reduce((acc, n) => acc + n, 0);
const has3 = nums.some(n => n === 3);
const allPos = nums.every(n => n > 0);
const firstEven = nums.find(n => n % 2 === 0);

// Spread / rest / desestructuración
const copy = [...nums, 6];
const [first, second, ...rest] = nums;

const user = { id: 1, name: 'Ana', role: 'admin' };
const { name, ...others } = user;
const updated = { ...user, name: 'Luis' }; // copia superficial

// Object helpers
Object.keys(user);
Object.values(user);
Object.entries(user).forEach(([k, v]) => console.log(k, v));
```

Inmutabilidad práctica: preferir `map` / `filter` y spread frente a `push` / `splice` cuando compartes estado (React, Redux, etc.). Recuerda: spread es *shallow*.

Ejercicios · Capítulo 1

1. Crea un módulo `storage.js` que exporte una factory `createStore(initial)` con `get`, `set` y `subscribe` usando closures.
2. Escribe `async function withRetry(fn, times)` que reintente una función async si falla, con delay fijo entre intentos.
3. Dado un array de usuarios `{id, active, score}`, obtén los activos, ordénalos por score descendente y devuelve solo sus ids (encadena `filter` / `sort` / `map`).

2. Operadores modernos de JavaScript

Estos operadores reducen código defensivo y clarifican *cuándo* debe aplicarse un valor por defecto. Dominar el short-circuit evita bugs sutiles con `0`, `''` y `false`.

2.1 Optional chaining (`?.`)

Accede a propiedades/métodos anidados sin lanzar si algo es `null` o `undefined`. El resultado corto es `undefined`.

```
const data = {
  user: {
    profile: { email: 'a@b.com' },
    prefs: null,
  },
};

console.log(data.user?.profile?.email); // 'a@b.com'
console.log(data.user?.prefs?.theme);   // undefined (no Error)
console.log(data.admin?.name);          // undefined

// También con llamadas y arrays
const fn = data.user?.save?.(); // solo llama si save existe
const item = data.list?.[0];

// No uses ?. para “silenciar” errores de lógica: oculta bugs
```

2.2 Nullish coalescing (`??`)

Devuelve el lado derecho solo si el izquierdo es `null` o `undefined`. A diferencia de `||`, **no** trata `0`, `''` ni `false` como “faltantes”.

```
const pageSize = 0;

console.log(pageSize || 10); // 10  ← ¡bug! 0 es válido
console.log(pageSize ?? 10); // 0   ← correcto

const title = '';
console.log(title || 'Sin título'); // 'Sin título'
console.log(title ?? 'Sin título'); // ''  (cadena vacía se respeta)

const name = null;
console.log(name ?? 'Anónimo'); // 'Anónimo'
```

2.3 Short-circuit con `&&` y `||`

```
// || : primer valor "truthy"
const host = process.env.HOST || 'localhost';

// && : si izq es truthy, evalúa der (útil en JSX antiguo)
user && user.login();

// Combinación típica segura:
const theme = settings?.theme ?? 'light';
const label = (count ?? 0) > 0 && `${count} items`;
```

2.4 Logical assignment

```
let options = { retries: 0 };

// ||= asigna solo si el valor actual es falsy
options.debug ||= true;      // si debug era undefined/''/0/false → true

// &&= asigna solo si el valor actual es truthy
options.cache &&= loadCache();

// ??= asigna solo si null/undefined (preferido para defaults)
options.retries ??= 3;       // retries era 0 → se queda en 0
options.timeout ??= 1000;    // timeout undefined → 1000
```

Cuándo usar cada fallback:

- `??` / `??=` : valores numéricos, booleanos o strings donde 0/false/'' son válidos.
- `||` / `||=` : cuando quieres "cualquier cosa vacía → default" (menos preciso).
- `?.` : navegación segura; combínalo con `??` para default final.

Ejercicios · Capítulo 2

1. Escribe `getConfig(cfg)` que lea `cfg?.server?.port` con default 3000, y `cfg?.server?.host` con default `'0.0.0.0'`, respetando puerto `0`.
2. Explica (con ejemplo) por qué `value || 'default'` falla para un contador que puede ser 0.
3. Usa `??=` para inicializar `map[key]` a un array vacío solo si aún no existe.

3. Tipos primitivos y especiales en TypeScript

TypeScript añade un sistema de tipos *estático* que se borra al compilar a JavaScript. En compile-time te avisa de errores; en runtime solo queda JS. Por eso los tipos no validan datos de una API por sí solos.

3.1 Primitivos

```
let name: string = 'Ada';
let age: number = 36;       // no hay int/float separados
let active: boolean = true;
let nothing: null = null;
let missing: undefined = undefined;

// En strictNullChecks, null y undefined no son asignables a string
// let s: string = null; // Error con strict
```

3.2 `void`, `never`, `any`, `unknown`

Tipo	Significado	Uso típico
`void`	Función que no retorna valor útil	callbacks, efectos
`never`	No produce valor: error o loop infinito	exhaustiveness checks
`any`	Desactiva el chequeo de tipos	evitar; solo migración temporal
`unknown`	“Algo”, pero debes estrechar antes de usarlo	entrada externa tipada con seguridad

```ts
function log(msg: string): void {
  console.log(msg);
}

function fail(message: string): never {
  throw new Error(message);
}

function infinite(): never {
  while (true) {}
}

// any: pierdes seguridad
function bad(x: any) {
  return x.toUpperCase(); // TS no se queja aunque x sea number
}

// unknown: obliga a narrowing
function good(x: unknown) {
  if (typeof x === 'string') {
    return x.toUpperCase(); // OK: ya sabemos que es string
  }
  throw new Error('se esperaba string');
}
```

Por qué `any` debilita la seguridad: contagia tipado "apagado" a todo lo que toca. Prefiere `unknown` + narrowing, o tipos precisos. En `tsconfig`, activa `noImplicitAny` (incluido en `strict`).

Ejercicios · Capítulo 3

1. Escribe `assertNever(x: never)` y úsalo en un `switch` para forzar cubrir todos los casos de una unión.
2. Convierte una función que recibe `any` a `unknown` y añade el narrowing necesario.
3. ¿Qué tipo de retorno tiene `() => { throw new Error() }`? Demuéstralo con anotación explícita `never`.

4. Arrays y tuplas

4.1 Arrays tipados

```
const scores: number[] = [10, 9, 8];
const names: Array<string> = ['Ana', 'Luis']; // equivalente

// Readonly: no push/splice/asignación a índice (en tipos)
const frozen: ReadonlyArray<number> = [1, 2, 3];
// frozen.push(4); // Error de tipos

function sumAll(xs: readonly number[]): number {
  return xs.reduce((a, b) => a + b, 0);
}
```

4.2 Tuplas: longitud y posiciones fijas

Una tupla tipa cada posición. Ideal para pares `[x, y]`, respuestas `[data, error]`, o argumentos rest tipados.

```
type Point = [number, number];
const p: Point = [10, 20];
const x = p[0]; // number
const y = p[1]; // number
// const z = p[2]; // Error: índice 2 no existe

// Opcional y rest en tuplas
type Row = [string, number?, ...boolean[]];
const r1: Row = ['id'];
const r2: Row = ['id', 1, true, false];

// Tupla readonly
type Pair = readonly [string, number];
const pair: Pair = ['age', 30];
// pair[0] = 'x'; // Error

// Desestructuración segura
function distance([x1, y1]: Point, [x2, y2]: Point): number {
  return Math.hypot(x2 - x1, y2 - y1);
}
```

Array vs tupla: `number[]` tiene longitud desconocida y todos los elementos son number. `[number, number]` tiene exactamente 2 números en orden. Usa tuplas cuando la posición tiene significado semántico distinto.

Ejercicios · Capítulo 4

1. Tipa una función `div(a, b)` que retorne `[result, error]` donde uno de los dos es `null`.
2. Crea el tipo `HttpHeader = readonly [string, string]` y una función que los convierta a objeto `Record<string, string>`.
3. Explica qué rompe si usas `string[]` donde necesitabas `[string, string]`.

5. Interfaces y type aliases

TypeScript usa **tipado estructural** (duck typing): si un valor tiene la forma requerida, es compatible, aunque no declare explícitamente la interfaz.

5.1 type vs interface

```
interface User {
  id: number;
  name: string;
}

type UserId = number; // alias de primitivo: solo con type
type Admin = User & { permissions: string[] }; // intersección

// Extender
interface Employee extends User {
  department: string;
}

type EmployeeT = User & { department: string }; // similar con type

// Tipado estructural
const u = { id: 1, name: 'Ada', extra: true };
const asUser: User = u; // OK: tiene al menos id y name
```

5.2 Declaration merging (solo interfaces)

```
interface WindowConfig {
  title: string;
}
interface WindowConfig {
  darkMode?: boolean; // se fusiona
}
const cfg: WindowConfig = { title: 'App', darkMode: true };

// type NO se fusiona:
// type A = { x: number };
// type A = { y: number }; // Error
```

5.3 Cuándo elegir cada uno

- **interface:** objetos públicos extensibles, libs, merging, estilo OOP.
- **type:** uniones, tuplas, mapped types, aliases de primitivos, composiciones complejas.
- En un proyecto, sé consistente; ambos son válidos para formas de objeto simples.

Ejercicios · Capítulo 5

1. Modela `Product`, `DigitalProduct` y `PhysicalProduct` con extends o intersecciones.
2. Demuestra tipado estructural con un objeto literal que satisfaga una interfaz sin implementarla.
3. Intenta fusionar dos `type` con el mismo nombre y documenta el error del compilador.

6. Enums y tipos literales

6.1 Literal unions

```
type Status = 'success' | 'error' | 'loading';

function setStatus(s: Status) { /* ... */ }
setStatus('success');
// setStatus('ok'); // Error

type HttpCode = 200 | 404 | 500;
```

6.2 Enums numéricos y de string

```
enum Direction {
  Up,    // 0
  Down,  // 1
  Left,  // 2
  Right, // 3
}

enum Color {
  Red = 'RED',
  Blue = 'BLUE',
}

const d: Direction = Direction.Up;
```

> **Tradeoff:** los enums numéricos son poco transparentes en JS emitido y permiten acceso inverso confuso. Muchos equipos prefieren `as const` + unión de literales.

6.3 Alternativa moderna: `as const`

```
const Status = {
  Success: 'success',
  Error: 'error',
  Loading: 'loading',
} as const;

type Status = typeof Status[keyof typeof Status];
// 'success' | 'error' | 'loading'

function paint(s: Status) {}
paint(Status.Success);
```

Ventajas de `as const`: objeto real en runtime, sin codegen raro de enum, tipado estrecho, fácil de serializar a JSON. Usa enum solo si tu dominio/equipo ya lo estandarizó.

Ejercicios · Capítulo 6

1. Reemplaza un enum de roles por un objeto `as const` y deriva el tipo unión.
2. Crea un tipo `Result = { ok: true; data: T } | { ok: false; error: string }` (prepárate para discriminated unions del cap. 10).
3. Lista pros/contras de enum string vs literal union en una tabla de 4 filas.

7. Inferencia y aserciones de tipos

7.1 Inferencia y tipado contextual

```
const n = 42;          // inferido: number
const label = 'hola';  // inferido: string
const list = [1, 2];   // number[]

// Contextual typing: el callback hereda el tipo del parámetro esperado
const nums = [1, 2, 3];
nums.forEach(x => {
  // x es number sin anotarlo
  console.log(x.toFixed(1));
});

const btn = document.querySelector('button');
btn?.addEventListener('click', (event) => {
  // event inferido como MouseEvent
  console.log(event.clientX);
});
```

7.2 Aserciones `as` y por qué no validan en runtime

```
const el = document.getElementById('app') as HTMLDivElement;
// Le dices a TS: “confía en mí”. Si el HTML no tiene #app o no es div,
// en runtime puede ser null → crash. La aserción NO convierte ni valida.

const data = JSON.parse(raw) as User; // peligroso sin validar forma

// as const: estrecha a literales
const roles = ['admin', 'user'] as const;
// readonly ['admin', 'user']
```

7.3 `satisfies`

Comprueba que un valor encaja en un tipo *sin ensanchar* la inferencia (conserva literales).

```
type Palette = Record<string, string>;

const colors = {
  primary: '#0f766e',
  danger: '#b91c1c',
} satisfies Palette;

// colors.primary es '#0f766e' (literal), no solo string
// Si escribieras : Palette, perderías los literales
```

Regla: aserciones son un escape hatch. Prefiere narrowing, validación runtime, o genéricos. `satisfies` valida la forma y mantiene precisión.

Ejercicios · Capítulo 7

1. Compara tipar un objeto con `: Palette` vs `satisfies Palette` y muestra la diferencia en el tipo de una propiedad.
2. Encuentra un caso donde `as` oculte un bug y reescríbelo con narrowing.
3. Explica en una frase por qué `as User` tras `JSON.parse` no garantiza un User real.

8. Funciones y genéricos

8.1 Parámetros y retornos tipados

```
function add(a: number, b: number): number {
  return a + b;
}

const greet = (name: string, shout?: boolean): string =>
  shout ? `¡HOLA ${name}!` : `Hola ${name}`;

// Parámetro rest
function sum(...xs: number[]): number {
  return xs.reduce((a, b) => a + b, 0);
}
```

8.2 Overloads

```
function parse(input: string): User;
function parse(input: string, safe: true): User | null;
function parse(input: string, safe?: boolean): User | null {
  try {
    return JSON.parse(input) as User; // en prod: validar
  } catch {
    if (safe) return null;
    throw new Error('JSON inválido');
  }
}
```

8.3 Genéricos

Un genérico parametriza un tipo para reutilizar lógica sin perder precisión. Piensa en él como una variable de tipo.

```ts
function identity<T>(value: T): T {
  return value;
}
const s = identity('hola'); // string
const n = identity(42);     // number

function first<T>(arr: T[]): T | undefined {
  return arr[0];
}

// Constraints
function longest<T extends { length: number }>(a: T, b: T): T {
  return a.length >= b.length ? a : b;
}
longest('ab', 'a');       // OK
longest([1, 2], [1]);     // OK
// longest(10, 20);       // Error: number no tiene length

// Interfaces / clases genéricas
interface Repo<T> {
  get(id: string): Promise<T | null>;
  save(entity: T): Promise<void>;
}

class Box<T> {
  constructor(public value: T) {}
  map<U>(fn: (v: T) => U): Box<U> {
    return new Box(fn(this.value));
  }
}
```

`T extends { length: number }`: restringe T a tipos que tengan `length`. Así puedes usar esa propiedad con seguridad dentro de la función.

Ejercicios · Capítulo 8

1. Implementa `pluck<T, K extends keyof T>(obj: T, key: K): T[K]`.
2. Escribe una clase `Stack<T>` con `push`, `pop`, `peek`.
3. Crea overloads para `format` que acepte `Date` o `string` ISO y retorne string.

9. Uniones e intersecciones

9.1 Unión `|` : una u otra

```
type Id = string | number;

function printId(id: Id) {
  // Solo puedes usar lo común a string y number… casi nada
  console.log(String(id));
  if (typeof id === 'string') {
    console.log(id.toUpperCase()); // narrowing
  }
}

type Success = { ok: true; data: string };
type Failure = { ok: false; error: string };
type Result = Success | Failure;
```

9.2 Intersección `&` : ambas a la vez

```
type Timestamped = { createdAt: Date };
type Author = { authorId: string };
type Post = { title: string; body: string };

type BlogPost = Post & Timestamped & Author;

const p: BlogPost = {
  title: 'TS',
  body: '...',
  createdAt: new Date(),
  authorId: 'u1',
};

// Cuidado: intersecar primitivos incompatibles → never
type Impossible = string & number; // never
```

> **Intuición:** `|` amplía posibilidades (OR); `&` acumula requisitos (AND) sobre objetos compatibles.

Ejercicios · Capítulo 9

1. Define `ApiResponse<T> = { data: T } | { error: string }` y una función que la consuma con narrowing.
2. Combina `WithId` y `WithName` vía intersección para un tipo `Entity`.
3. ¿Qué tipo resulta de `{ a: string } & { a: number }`? Experimenta y explica.

10. Type narrowing y guards

Narrowing es el proceso por el cual TypeScript reduce una unión a un miembro más específico tras un chequeo en runtime.

10.1 Técnicas built-in

```
function padLeft(value: string | number) {
  if (typeof value === 'number') {
    return ' '.repeat(value);
  }
  return value; // string
}

function printDate(d: Date | string) {
  if (d instanceof Date) {
    console.log(d.toISOString());
  } else {
    console.log(d.toUpperCase());
  }
}

type Fish = { swim: () => void };
type Bird = { fly: () => void };

function move(animal: Fish | Bird) {
  if ('swim' in animal) animal.swim();
  else animal.fly();
}

function handle(x: string | null) {
  if (x === null) return;
  console.log(x.length); // string
}
```

10.2 Type predicates

```
function isFish(a: Fish | Bird): a is Fish {
  return (a as Fish).swim !== undefined;
}

function go(a: Fish | Bird) {
  if (isFish(a)) a.swim();
  else a.fly();
}
```

10.3 Discriminated unions

```
type Shape =
  | { kind: 'circle'; radius: number }
  | { kind: 'rect'; width: number; height: number };

function area(s: Shape): number {
  switch (s.kind) {
    case 'circle':
      return Math.PI * s.radius ** 2;
    case 'rect':
      return s.width * s.height;
    default: {
      const _exhaustive: never = s;
      return _exhaustive;
    }
  }
}
```

Discriminante: una propiedad común (`kind`) con literales distintos permite al compilador saber qué campos existen en cada rama. Es el patrón más limpio para modelos de dominio.

Ejercicios · Capítulo 10

1. Modela eventos de UI con discriminated union (`click | keydown | submit` y una función `handle(event)` .
2. Escribe el predicate `isStringArray(x: unknown): x is string[]` .
3. Añade un nuevo shape `triangle` y observa cómo `never` te obliga a actualizar `area` .

11. Utility types

TypeScript incluye tipos utilitarios basados en mapped/conditional types. Úsalos antes de inventar los tuyos.

```
interface User {
  id: number;
  email: string;
  name: string;
  password: string;
}

type UserPatch = Partial<User>;           // todo opcional (PATCH)
type RequiredUser = Required<UserPatch>;  // todo requerido
type PublicUser = Omit<User, 'password'>; // sin password
type Creds = Pick<User, 'email' | 'password'>;
type ReadonlyUser = Readonly<User>;

type Roles = 'admin' | 'editor' | 'viewer';
type Permissions = Record<Roles, boolean>;

function createUser(email: string, name: string) {
  return { id: 1, email, name, password: 'x' };
}
type Created = ReturnType<typeof createUser>;
type Args = Parameters<typeof createUser>; // [string, string]
```

11.1 Mapped y conditional (cuándo custom)

```
// Mapped: transformar propiedades
type Nullable<T> = { [K in keyof T]: T[K] | null };

// Conditional: elige según un test de tipos
type IsString<T> = T extends string ? true : false;

// Solo escribe custom cuando los utility types no bastan
// (p.ej. DeepReadonly, paths de objeto, infer en cadenas de templates)
```

> **Guía rápida:** Partial (updates), Pick/Omit (DTOs), Record (mapas de claves conocidas), ReturnType/Parameters (derivar de funciones existentes sin duplicar).

Ejercicios · Capítulo 11

1. Parte de `User`, define `UserDTO` sin password y con email readonly.
2. Crea `Optional<T, K extends keyof T>` que haga opcionales solo las claves K (mapped type).
3. Usa `ReturnType` sobre una función async y observa que obtienes `Promise<...>`; extrae el T interno con un conditional type `Await<T>`.

12. DOM APIs tipadas

El tipado del DOM asume un documento ideal. En runtime el markup puede no coincidir: siempre estrecha valores nullable.

```
// querySelector puede devolver null
const el = document.querySelector('#app');
if (!el) throw new Error('#app no existe');

// Estrechar al elemento concreto
const input = document.querySelector<HTMLInputElement>('#email');
if (input) {
  console.log(input.value); // HTMLInputElement
}

const form = document.getElementById('signup');
if (form instanceof HTMLFormElement) {
  form.addEventListener('submit', (event: SubmitEvent) => {
    event.preventDefault();
    const data = new FormData(form);
    console.log(data.get('email'));
  });
}

// Eventos: tipa el currentTarget si hace falta
button.addEventListener('click', (e) => {
  const btn = e.currentTarget;
  if (btn instanceof HTMLButtonElement) {
    btn.disabled = true;
  }
});
```

> **Gap compile-time vs runtime:** `as HTMLInputElement` no comprueba el tag real. Prefiere `instanceof`, generics de `querySelector`, o guards. Los tipos del DOM vienen de `lib.dom.d.ts`.

Ejercicios · Capítulo 12

1. Escribe `mustGet<T extends Element>(sel: string): T` que lance si no hay match.
2. Tipa un listener de `input` que lea `event.currentTarget.value` con seguridad.
3. Explica por qué `document.getElementById` retorna `HTMLElement | null` y no el tipo del tag.

13. Configuración del proyecto TypeScript

13.1 `tsconfig.json` esencial

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "lib": ["ES2020", "DOM"],
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "skipLibCheck": true,
    "esModuleInterop": true,
    "resolveJsonModule": true,
    "declaration": true,
    "declarationMap": true,
    "sourceMap": true,
    "outDir": "dist",
    "rootDir": "src"
  },
  "include": ["src"]
}
```

13.2 Strict mode

`"strict": true` activa: `strictNullChecks`, `noImplicitAny`, `strictFunctionTypes`, etc. Es la base de un tipado útil. Sin strict, TypeScript pierde gran parte de su valor.

13.3 Declaration files (`.d.ts`)

```
// types/vendor.d.ts
declare module 'old-lib' {
  export function doThing(x: string): number;
}

// ambient para globals
declare const APP_VERSION: string;
```

13.4 Migración incremental JS → TS

1. Activa `allowJs` y `checkJs` (opcional al inicio).
2. Renombra archivos a `.ts` / `.tsx` por carpetas.
3. Usa `// @ts-check` en JS críticos.
4. Reemplaza `any` gradualmente; no bloquees el merge por perfección.
5. Sube strictness con flags uno a uno si el salto a strict es brutal.

Ejercicios · Capítulo 13

1. Crea un `tsconfig.json` strict para una app de navegador con `DOM` y otro para Node sin DOM.
2. Añade un `.d.ts` para una librería sin tipos.
3. Explica la diferencia entre `module: CommonJS` y `ESNext` en el emit.

14. Validación en runtime

Los tipos de TypeScript **desaparecen** al compilar. Un `User` tipado tras `fetch` es solo una promesa al compilador: el JSON puede traer campos faltantes, tipos incorrectos o inyección inesperada.

```
// ❌ Falsa seguridad
async function loadUser(url: string): Promise<User> {
  const res = await fetch(url);
  return res.json(); // Promise<any> → aserción implícita peligrosa
}

// ✅ Validar forma en runtime (ejemplo manual)
function isUser(data: unknown): data is User {
  if (typeof data !== 'object' || data === null) return false;
  const u = data as Record<string, unknown>;
  return typeof u.id === 'number' && typeof u.name === 'string';
}

async function loadUserSafe(url: string): Promise<User> {
  const res = await fetch(url);
  const data: unknown = await res.json();
  if (!isUser(data)) throw new Error('payload inválido');
  return data;
}

// Con librería (Zod) — patrón recomendado en apps reales
// const UserSchema = z.object({ id: z.number(), name: z.string() });
// type User = z.infer<typeof UserSchema>;
// const user = UserSchema.parse(await res.json());
```

> **Fronteras de confianza:** valida en los bordes del sistema (HTTP, localStorage, formularios, postMessage). Dentro de tu código tipado, confía en los tipos; en el borde, no.

Ejercicios · Capítulo 14

1. Escribe un guard para `{ items: string[]; total: number }`.
2. Simula un API que a veces manda `total` como string y muestra cómo el guard lo rechaza.
3. Investiga Zod o Valibot y reescribe el guard con un schema (opcional, en tu máquina).

15. Errores, inmutabilidad y funciones pequeñas

15.1 Manejo de errores

```
class AppError extends Error {
  constructor(
    message: string,
    public code: string,
    public cause?: unknown,
  ) {
    super(message);
    this.name = 'AppError';
  }
}

async function readConfig(path: string) {
  try {
    const text = await readFile(path);
    return parseConfig(text);
  } catch (err) {
    throw new AppError('No se pudo leer config', 'CONFIG_READ', err);
  }
}

// Resultado explícito (sin excepciones) para flujos esperables
type Result<T, E = Error> =
  | { ok: true; value: T }
  | { ok: false; error: E };

function divide(a: number, b: number): Result<number, string> {
  if (b === 0) return { ok: false, error: 'división por cero' };
  return { ok: true, value: a / b };
}
```

15.2 Inmutabilidad

```
const state = { items: [1, 2], user: { name: 'Ana' } };

// Mutación (evita compartir este objeto en varios sitios)
state.items.push(3);

// Actualización inmutable (shallow)
const next = {
  ...state,
  items: [...state.items, 3],
  user: { ...state.user, name: 'Luis' },
};

// Object.freeze es shallow
Object.freeze(state);
```

15.3 Funciones pequeñas y composables

```
const trim = (s: string) => s.trim();
const lower = (s: string) => s.toLowerCase();
const slugify = (s: string) =>
  s.replace(/\s+/g, '-').replace(/[^a-z0-9-]/g, '');

// Composición simple
const toSlug = (s: string) => slugify(lower(trim(s)));

// pipe genérico
const pipe = <T>(...fns: Array<(v: T) => T>) =>
  (value: T) => fns.reduce((acc, fn) => fn(acc), value);

const normalize = pipe(trim, lower, slugify);
console.log(normalize('  Hola Mundo  ')); // hola-mundo
```

Ideas clave: una función = una responsabilidad; inputs claros; preferir retornar valores nuevos; errores de dominio tipados; composición frente a funciones monstruo de 200 líneas.

Ejercicios · Capítulo 15

1. Implementa `Result` helpers: `map`, `flatMap`, `unwrapOr`.
2. Reescribe un update de lista de todos sin mutar el array ni los objetos internos.
3. Parte una función “dios” de validación+guardado+log en 3 funciones composables.

16. Estructuras de datos en JavaScript

Elige la estructura según el **patrón de acceso**: ¿buscas por clave? ¿necesitas unicidad? ¿FIFO? ¿relaciones?

16.1 Array

Índices numéricos, orden, acceso O(1) por índice, búsqueda por valor O(n).

```
const q = ['a', 'b'];
q.push('c');        // al final
q.pop();            // quita final
q.unshift('z');     // al inicio (O(n))
q.shift();          // quita inicio (O(n))
```

16.2 Object y Map

```
// Object: claves string/symbol, prototipo, ideal para records fijos
const user = { id: 1, name: 'Ada' };

// Map: cualquier clave, tamaño confiable, iteración de inserción
const visits = new Map<string, number>();
visits.set('home', 1);
visits.set('home', (visits.get('home') ?? 0) + 1);
console.log(visits.size);
for (const [k, v] of visits) console.log(k, v);

// Usa Map cuando las claves son dinámicas, no-string, o necesitas size limpio
```

16.3 Set

```
const tags = new Set<string>(['js', 'ts', 'js']);
tags.add('node');
console.log(tags.has('ts')); // true
console.log(tags.size);      // 3 (sin duplicados)
const unique = [...new Set([1, 1, 2, 3])];
```

16.4 Stack y Queue

```
// Stack (LIFO) con array
class Stack<T> {
  private data: T[] = [];
  push(v: T) { this.data.push(v); }
  pop(): T | undefined { return this.data.pop(); }
  peek(): T | undefined { return this.data[this.data.length - 1]; }
  get size() { return this.data.length; }
}

// Queue (FIFO): shift es O(n); para colas grandes usa linked list o deque
class Queue<T> {
  private data: T[] = [];
  enqueue(v: T) { this.data.push(v); }
  dequeue(): T | undefined { return this.data.shift(); }
}
```

16.5 Árboles y grafos (idea)

```
type TreeNode<T> = {
  value: T;
  children: TreeNode<T>[];
};

// Grafo dirigido por lista de adyacencia
type Graph = Map<string, string[]>;

const g: Graph = new Map([
  ['A', ['B', 'C']],
  ['B', ['D']],
  ['C', ['D']],
  ['D', []],
]);

function bfs(start: string, graph: Graph): string[] {
  const seen = new Set<string>([start]);
  const queue = [start];
  const order: string[] = [];
  while (queue.length) {
    const node = queue.shift()!;
    order.push(node);
    for (const next of graph.get(node) ?? []) {
      if (!seen.has(next)) {
        seen.add(next);
        queue.push(next);
      }
    }
  }
  return order;
}
```

Necesitas...	Usa
Lista ordenada / stack simple	Array
Lookup por id dinámico	Map (o object si claves fijas)
Unicidad / membership	Set
Jerarquía (DOM, files)	Árbol
Relaciones / rutas	Grafo

Ejercicios · Capítulo 16

1. Implementa un LRU cache mínimo con `Map` (orden de inserción).
2. Escribe DFS recursivo sobre el grafo de ejemplo.
3. Dado un array con duplicados, mide mentalmente por qué `Set` es mejor que un doble bucle para unicidad.

17. Análisis de complejidad (Big-O)

La notación Big-O describe cómo crece el costo (tiempo/espacio) cuando crece la entrada n. Nos importa el orden de magnitud, no los milisegundos exactos.

Complejidad	Nombre	Ejemplo típico
O(1)	Constante	acceso `arr[i]`, `Map.get`
O(log n)	Logarítmica	binary search
O(n)	Lineal	`find`, un for sobre n
O(n log n)	Linealítmica	`sort` eficiente
O(n^2)	Cuadrática	doble bucle anidado

```
// O(n): un recorrido
function includes(arr: number[], target: number): boolean {
  for (const x of arr) if (x === target) return true;
  return false;
}

// O(n²): anidados
function hasDuplicateNaive(arr: number[]): boolean {
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      if (arr[i] === arr[j]) return true;
    }
  }
  return false;
}

// O(n) tiempo, O(n) espacio: hash set
function hasDuplicate(arr: number[]): boolean {
  const seen = new Set<number>();
  for (const x of arr) {
    if (seen.has(x)) return true; // O(1) promedio
    seen.add(x);
  }
  return false;
}

// Ordenar: típicamente O(n log n)
const sorted = [...arr].sort((a, b) => a - b);

// Lookup: Object/Map ≈ O(1) promedio; Array.includes ≈ O(n)
```

> **Espacio:** un `Set` / `Map` auxiliar suele ser O(n) espacio a cambio de tiempo. Pregúntate siempre: ¿el cuello de botella es CPU o memoria? ¿n es 100 o 10^7?

Ejercicios · Capítulo 17

1. Clasifica: sumar un array; ordenar y luego binary search; tres bucles anidados independientes vs anidados.
2. Reescribe un conteo de frecuencias de $O(n^2)$ a $O(n)$ con Map.
3. ¿Cuál es el costo de `unshift` repetido n veces en un array? ¿Cómo lo evitas?

18. Soluciones orientativas

No son las únicas respuestas correctas: sirven para comparar tu enfoque. Intenta primero sin mirar.

Cap. 1 · createStore (idea)

```
export function createStore(initial) {
  let state = initial;
  const listeners = new Set();
  return {
    get: () => state,
    set: (next) => {
      state = typeof next === 'function' ? next(state) : next;
      listeners.forEach(fn => fn(state));
    },
    subscribe: (fn) => {
      listeners.add(fn);
      return () => listeners.delete(fn);
    },
  };
}
```

Cap. 2 · getConfig

```
function getConfig(cfg) {
  return {
    port: cfg?.server?.port ?? 3000,
    host: cfg?.server?.host ?? '0.0.0.0',
  };
}
```

Cap. 3 · assertNever

```
function assertNever(x: never): never {
  throw new Error(`Unexpected: ${x}`);
}
```

Cap. 4 · div con tupla

```
function div(a: number, b: number): [number, null] | [null, string] {
  if (b === 0) return [null, 'división por cero'];
  return [a / b, null];
}
```

Cap. 8 · pluck

```
function pluck<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}
```

Cap. 10 · isStringArray

```
function isStringArray(x: unknown): x is string[] {
  return Array.isArray(x) && x.every(i => typeof i === 'string');
}
```

Cap. 11 · Optional<T, K>

```
type Optional<T, K extends keyof T> =
  Omit<T, K> & Partial<Pick<T, K>>;
```

Cap. 12 · mustGet

```
function mustGet<T extends Element>(
  sel: string,
  root: ParentNode = document,
): T {
  const el = root.querySelector<T>(sel);
  if (!el) throw new Error(`No match: ${sel}`);
  return el;
}
```

Cap. 14 · guard de lista

```
function isItemList(data: unknown): data is { items: string[]; total: number } {
  if (typeof data !== 'object' || data === null) return false;
  const d = data as Record<string, unknown>;
  return Array.isArray(d.items)
    && d.items.every(i => typeof i === 'string')
    && typeof d.total === 'number';
}
```

Cap. 16-17 · frecuencia O(n)

```
function frequencies(arr: string[]): Map<string, number> {
  const map = new Map<string, number>();
  for (const x of arr) map.set(x, (map.get(x) ?? 0) + 1);
  return map;
}
// unshift n veces ≈ O(n²); preferir push + reverse, o deque
```

> **Plan de 2 semanas (sugerido):** días 1–3 caps 1–2; días 4–7 caps 3–8; días 8–10 caps 9–13; días 11–12 caps 14–15; días 13–14 caps 16–17 + repaso de ejercicios fallidos. Escribe código todos los días, aunque sea 30 minutos.

Guía generada para estudio personal · JavaScript / TypeScript · Practica en el editor con `strict: true`.